

Universidad Tecnológica de Zinacantepec

Entornos Virtuales y Negocios Digitales

Programación Orientada a Objetos

Proyecto Final Unidad IV

Profesor: Ing. Carlos Daniel Estrada Montes de Oca

Presentado por:

Aldo Rafael García Arriaga

Ernesto Lopez Perez

Erick Osvaldo Herrera Bernal

EVN-3

14 ago 2026

Índice

Introducción	3
1. Planteamiento del Problema	4
2. Objetivos del Proyecto	5
Objetivo general	5
Objetivos específicos	5
3. Cronograma de Actividades	6
Lunes 10 de Agosto (Planeación y Documentación Inicial)	6
Martes 11 de Agosto (Adaptación de Base de Datos e Interfaz)	7
Miércoles 12 de Agosto (Manejo de Excepciones y Arquitectura)	7
Jueves 13 de Agosto (Pruebas y Documentación del Sistema)	8
Viernes 14 de Agosto (Cierre, Conclusiones y Entregables)	8
4. Solución Propuesta	9
1. Tecnologías y su Funcionalidad	9
2. Estructura y Funcionamiento del Programa	9
A. Gestión de Base de Datos.	10
B. Interfaz y Lógica de Usuario	10
5. Desarrollo del Proyecto	11
Operación: Agregar (Guardar)	12
Operación: Consultar (Mostrar y Seleccionar)	13
Operación: Actualizar	14
Operación: Eliminar	14
6. Conclusiones	16
Principales aprendizajes	16
Dificultades enfrentadas	16
Soluciones implementadas	16
Posibles mejoras para futuras versiones	17

Introducción

El proyecto que fue elegido fue de la cafetería de la escuela, ya que se ha visto que han hecho anotación en libretas o en hojas, por ello se les dificulta saber cuánto vendieron exactamente, que vendieron, cada cuánto les conviene que venga el proveedor, y lo más importante lo ganado y obtenido en ganancias

Por ello se propone un sistema que ayude a facilitar el trabajo y no solo eso también a tener un control más preciso y menos laborioso con la ayuda de dicho programa que se llevará a cabo con lo visto en clase para ayudar a entender cómo funciona realmente un negocio y que conlleva hacer un programa de dicho negocio

1. Planteamiento del Problema

La cafetería de la institución educativa requiere llevar un control adecuado de los productos que recibe, vende y mantiene disponibles. Actualmente, el registro del inventario y de las ventas se realiza principalmente mediante hojas, apuntes y notas, lo que puede dificultar la organización y consulta de la información.

Este método puede ocasionar problemas como la pérdida de registros, errores al realizar las cuentas, dificultad para conocer con exactitud las cantidades disponibles de cada producto y mayor tiempo dedicado a revisar las ventas realizadas durante el día. Asimismo, al no contar con un registro digital permanente, puede resultar complicado determinar cuánto se vendió, qué productos tienen mayor movimiento, qué productos necesitan ser reabastecidos y si las cantidades registradas coinciden con las existencias reales.

Por esta razón, se propone desarrollar una aplicación de escritorio utilizando Python, Programación Orientada a Objetos, Tkinter y SQLite, que permita administrar de manera organizada la información relacionada con el inventario y las ventas de la cafetería. El sistema permitirá registrar los productos que ingresan, actualizar las existencias conforme se realicen ventas y conservar la información en una base de datos para facilitar su consulta y seguimiento.

De esta manera, la información proporcionada por la cafetería permitirá identificar sus necesidades reales y diseñar un sistema funcional que contribuya a mejorar el control de productos, facilitar el registro de ventas y mantener la información organizada y disponible de forma permanente.

2. Objetivos del Proyecto

Objetivo general

Desarrollar una aplicación de escritorio para la cafetería de la institución educativa que permita gestionar de manera organizada y permanente el inventario y las ventas, mediante el uso de Python, Programación Orientada a Objetos, Tkinter y SQLite, con el propósito de reducir errores en los registros y facilitar el control de los productos disponibles y vendidos.

Objetivos específicos

1. Analizar el proceso actual utilizado por la cafetería para registrar sus productos, ventas y cuentas, mediante la colaboración del personal encargado.
2. Identificar las principales dificultades que presenta el uso de hojas, notas y registros manuales para el control del inventario y las ventas.
3. Diseñar una interfaz gráfica sencilla e intuitiva utilizando Tkinter que permita registrar, consultar, modificar y eliminar información relacionada con los productos.
4. Implementar una base de datos utilizando SQLite para almacenar de manera permanente la información del inventario y los registros de ventas.
5. Evaluar el funcionamiento del sistema con información proporcionada por la cafetería, verificando si las funciones desarrolladas son útiles y si el personal considera viable utilizar la aplicación como alternativa al registro manual.

3. Cronograma de Actividades

Para el desarrollo e implementación del sistema de inventario para la Cafetería de la UTZin, se estableció un plan de trabajo de cinco días, abarcando del lunes 10 de agosto al viernes 14 de agosto del presente año.

Dado que el código partió de un ejercicio base realizado previamente en clase, el esfuerzo de programación se enfocó en la adaptación del sistema al contexto de la cafetería, el renombrado exhaustivo de variables para darles sentido dentro del nuevo modelo de negocio, y la robustez del sistema mediante la implementación de bloques try/except. De manera paralela, se dividió la carga documental para cumplir con todos los requerimientos.

A continuación, se presenta la distribución detallada de las actividades y sus responsables:

Lunes 10 de Agosto (Planeación y Documentación Inicial)	
Fases y Actividades	Responsables
Redacción de la Introducción, Punto 1 (Planteamiento del problema) y Punto 2 (Objetivos).	Erick
Elaboración del Punto 3 (Cronograma de actividades).	Aldo Rafael García Arriaga
Investigación y repaso de los repositorios en GitHub.	Ernesto Lopez Perez

Martes 11 de Agosto (Adaptación de Base de Datos e Interfaz)

Fases y Actividades	Responsables
Modificación de la base de datos SQLite para estructurar el inventario de la cafetería.	Aldo Rafael García Arriaga
Refactorización del código base (renombrado de variables, clases y métodos).	Erick
Ajustes en la interfaz gráfica (Tkinter) al nuevo contexto. (renombrando y modificando widgets)	Ernesto Lopez Perez

Miércoles 12 de Agosto (Manejo de Excepciones y Arquitectura)

Fases y Actividades	Responsables
Insertión de bloques try/except en las funciones CRUD (Agregar, Consultar, Actualizar, Eliminar) para prevenir errores de usuario.	Aldo Rafael García Arriaga Erick
Redacción del Punto 4 (Solución propuesta), detallando la arquitectura, tecnologías y esquema de BD.	Ernesto Lopez Perez

Jueves 13 de Agosto (Pruebas y Documentación del Sistema)

Fases y Actividades	Responsables
Realización de pruebas funcionales y toma de capturas de pantalla del sistema en ejecución.	Aldo Rafael García Arriaga Ernesto Lopez Perez Erick
Redacción del Punto 5 (Desarrollo del proyecto), explicando el funcionamiento general y operaciones CRUD.	Ernesto Lopez Perez Erick

Viernes 14 de Agosto (Cierre, Conclusiones y Entregables)

Fases y Actividades	Responsables
Redacción del Punto 6 (Conclusiones), integrando aprendizajes y dificultades.	Aldo Rafael García Arriaga
Generación del ejecutable de la aplicación.	Ernesto Lopez Perez
Integración del documento PDF, consolidación del repositorio en GitHub y preparación para la defensa oral (Punto 7).	Aldo Rafael García Arriaga Ernesto Lopez Perez Erick

4. Solución Propuesta

Este proyecto es una aplicación de escritorio diseñada para gestionar el catálogo de productos de una cafetería. Su desarrollo se divide en cuatro pilares tecnológicos fundamentales que trabajan en conjunto para crear un programa funcional, persistente y fácil de usar.

A continuación, se detalla cómo se desarrolló el proyecto, la funcionalidad de cada tecnología y la estructura del código.

1. Tecnologías y su Funcionalidad

- Python: Es el lenguaje de programación principal. Actúa como el puente que conecta la interfaz gráfica con la base de datos. En este proyecto, Python se encarga de validar los datos y ejecutar las instrucciones lógicas mediante Programación Orientada a Objetos.
- SQLite: Es un motor de base de datos ligero que no requiere instalar un servidor externo; guarda toda la información en un archivo local llamado `catalogo.db`. Su función aquí es asegurar que los productos agregados (nombre, stock y precio) no se borren cuando se cierre el programa.
- Tkinter: Es la biblioteca estándar de Python para crear ventanas, botones y campos de texto. Su objetivo es que el usuario final no tenga que usar la consola de comandos, permitiéndole interactuar con el sistema a través de clics y formularios intuitivos.
- GitHub: Aunque no está escrito en el código directamente, GitHub es la plataforma donde se aloja el repositorio del proyecto. Sirve para llevar un historial de los cambios, respaldar el código en la nube y documentar el proyecto para futuras actualizaciones o presentaciones académicas.

2. Estructura y Funcionamiento del Programa

El código está estructurado de manera modular utilizando dos clases principales. Esto mantiene el código ordenado, separando la lógica de los datos de la parte visual.

A. Gestión de Base de Datos.

Esta clase es responsable de la persistencia de los datos. Al iniciar, verifica si existe la tabla productos y, si no, la crea automáticamente. Contiene los métodos para realizar las operaciones fundamentales del sistema CRUD (Crear, Leer, Actualizar y Eliminar):

- Agregar: Inserta un nuevo producto con su stock y precio.
- Leer: Extrae todos los registros guardados para visualizarlos.
- Actualizar: Modifica los datos de un producto existente usando su ID.
- Eliminar: Borra definitivamente un registro de la base de datos.

B. Interfaz y Lógica de Usuario

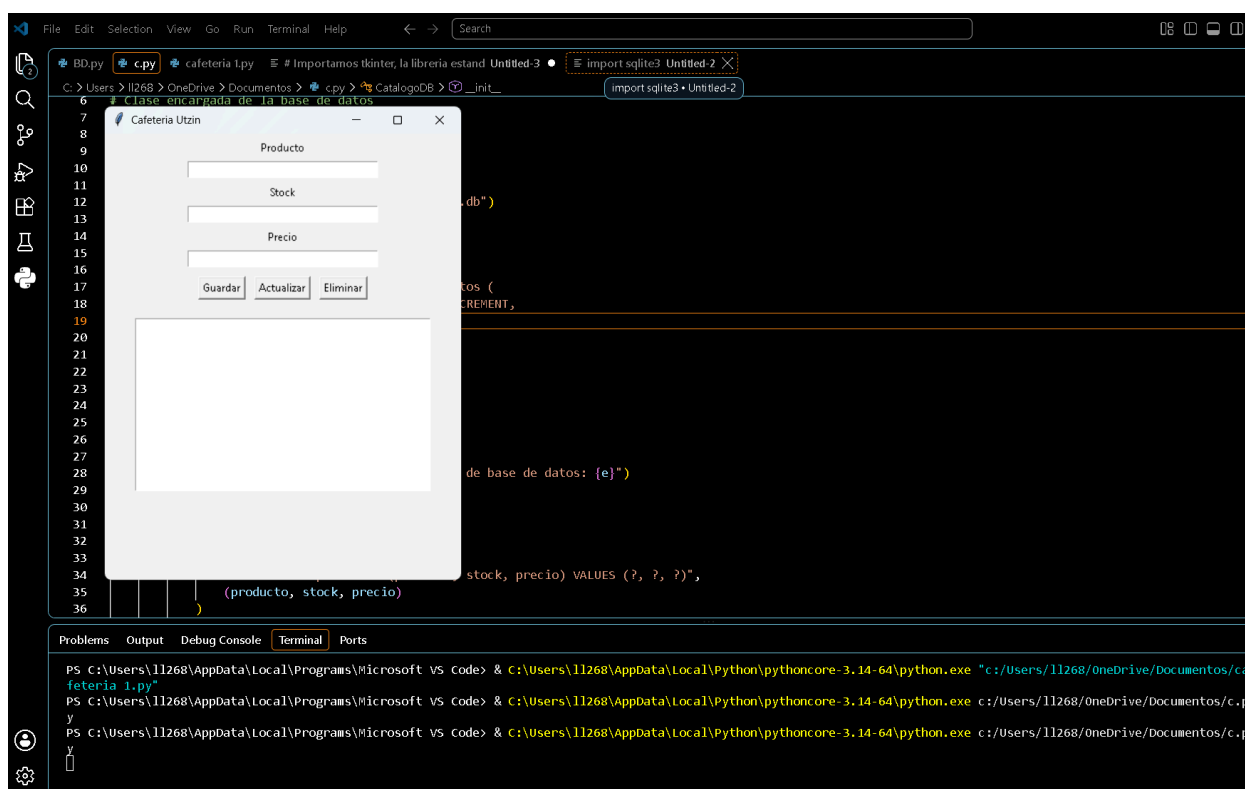
Esta clase construye la ventana principal y maneja la interacción directa con el usuario.

- Creación visual (crear interfaz): Dibuja las etiquetas, los cuadros de texto (Entry) para que el usuario escriba, los botones de acción y la lista interactiva (Listbox) donde se ven los productos.
- Validación de entrada (guardar, actualizar): Antes de enviar cualquier dato a SQLite, esta clase verifica que el usuario no haya dejado el nombre en blanco, que el stock sea un número entero y que el precio sea mayor a cero.
- Interactividad (seleccionar, limpiar): Detecta cuando el usuario hace clic en un producto de la lista, autocompletando los campos de texto para poder editarlo o eliminarlo fácilmente, y limpia los campos una vez terminada la acción.

5. Desarrollo del Proyecto

El sistema "Cafetería Utzin" es una aplicación de escritorio desarrollada en Python utilizando la biblioteca Tkinter para la interfaz gráfica y SQLite3 para la persistencia de datos.

Funcionamiento general de la aplicación Al iniciar el programa, la clase CafeteriaUtzinApp genera una ventana principal que integra un formulario de entrada (Producto, Stock, Precio), un panel de botones de acción y un cuadro de lista (Listbox) interactivo. En segundo plano, la clase CatalogoDB se conecta automáticamente al archivo local catalogo.db (o lo crea si no existe) y genera la tabla productos. El sistema mantiene sincronizada la interfaz visual con la base de datos, asegurando que cualquier cambio se refleje inmediatamente en la lista inferior.



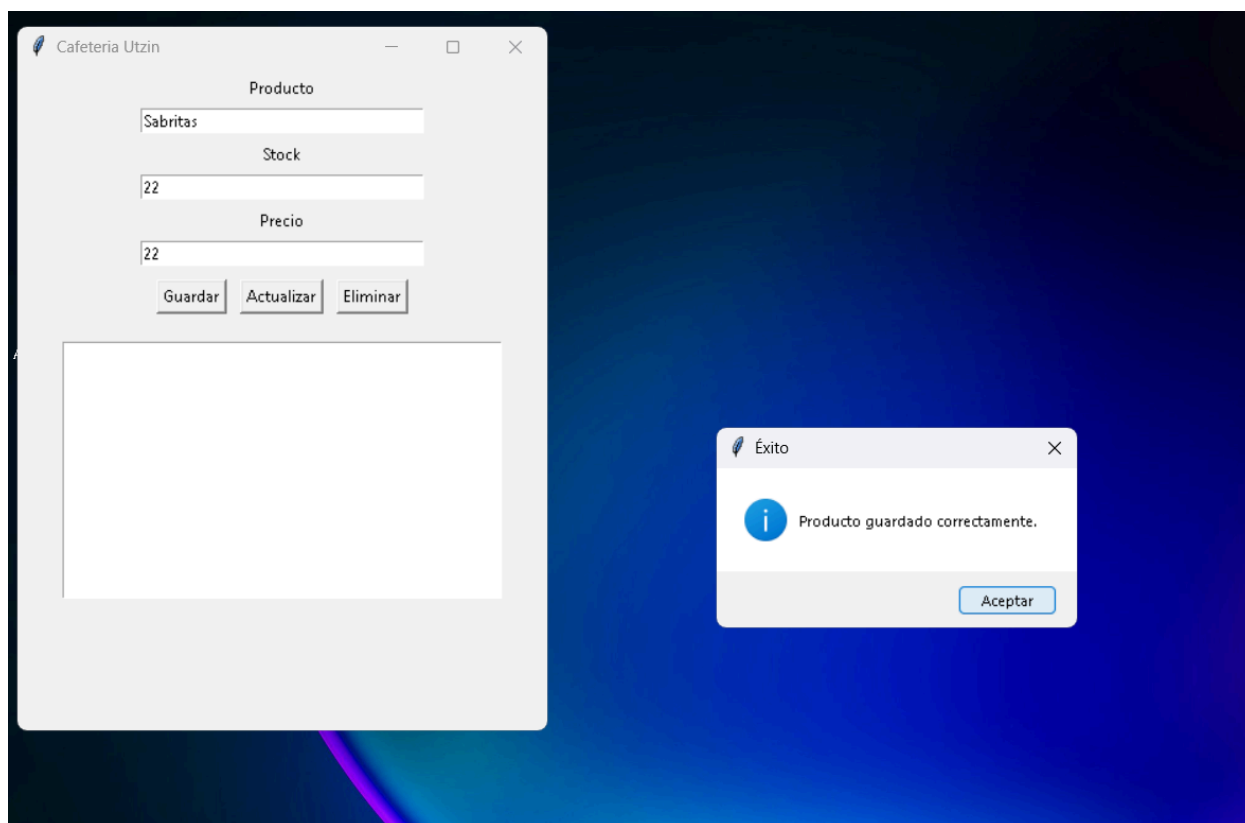
A continuación, se documentan las cuatro operaciones fundamentales (CRUD) implementadas en el sistema:

Operación: Agregar (Guardar)

Para registrar un nuevo elemento en el inventario, el usuario debe llenar los campos de texto correspondientes al nombre del producto, la cantidad en stock y su precio unitario. Al presionar el botón "Guardar", el sistema ejecuta el método guardar(), el cual realiza una serie de validaciones:

1. Comprueba que el campo del nombre no esté vacío.
2. Verifica que el stock sea un número entero y no sea negativo.
3. Asegura que el precio sea un valor numérico mayor a cero.

Si los datos son válidos, se llama al método agregar() de la base de datos, el cual ejecuta la sentencia SQL INSERT INTO productos.... Finalmente, se muestra una alerta de éxito, se limpian los campos y se actualiza la lista visible.



Operación: Consultar (Mostrar y Seleccionar)

La consulta de datos se realiza de forma automática y continua en el cuadro inferior (Listbox). El método `cargar_productos()` utiliza la instrucción SQL `SELECT * FROM productos` para extraer todos los registros de la base de datos y mostrarlos en formato de texto.

Además, se implementó un evento de selección (`<<ListboxSelect>>`). Cuando el usuario hace clic sobre un elemento de la lista, el método `seleccionar()` captura el ID de ese registro y rellena automáticamente los campos de texto superiores con la información del producto. Esto es el paso previo necesario para poder actualizar o eliminar.

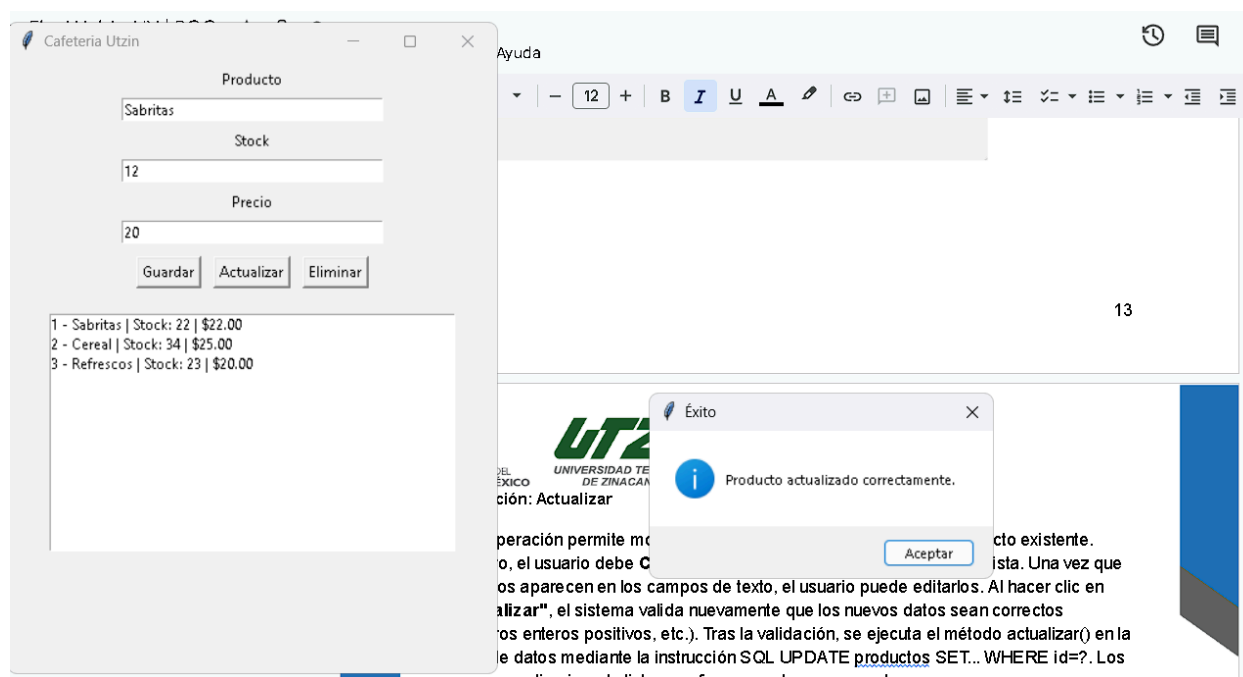


The screenshot shows a Windows application window titled "Cafeteria Utzin". The window contains a form with three text input fields labeled "Producto", "Stock", and "Precio". Below these fields are three buttons: "Guardar", "Actualizar", and "Eliminar". At the bottom of the window is a listbox containing the following text:

- 1 - Sabritas | Stock: 22 | \$22.00
- 2 - Cereal | Stock: 34 | \$25.00
- 3 - Refrescos | Stock: 23 | \$20.00

Operación: Actualizar

Esta operación permite modificar el stock, precio o nombre de un producto existente. Primero, el usuario debe Consultar/Seleccionar el producto desde la lista. Una vez que los datos aparecen en los campos de texto, el usuario puede editarlos. Al hacer clic en "Actualizar", el sistema valida nuevamente que los nuevos datos sean correctos (números enteros positivos, etc.). Tras la validación, se ejecuta el método actualizar() en la base de datos mediante la instrucción SQL UPDATE productos SET... WHERE id=?. Los campos se limpian y la lista se refresca con los nuevos valores.



Operación: Eliminar

Para retirar un producto del catálogo, el usuario lo selecciona desde la lista interactiva. Al presionar el botón "Eliminar", el método eliminar() primero lanza un cuadro de diálogo (messagebox.askyesno) pidiendo al usuario confirmar la acción para evitar borrados accidentales. Si el usuario confirma, el sistema ejecuta la instrucción SQL DELETE FROM productos WHERE id=? borrando el registro de forma permanente. La interfaz se limpia y la lista de inventario se recarga, ya sin el producto eliminado.

Cafeteria Utzin

Producto

Cereal

Stock

34

Precio

25.0

Guardar Actualizar Eliminar

1 - Sabritas	Stock: 12	\$20.00
2 - Cereal	Stock: 34	\$25.00
3 - Refrescos	Stock: 23	\$20.00

Ayuda

ntales. Si el usuario confirma, el sistema ejecuta la instrucción SQL DELETE productos WHERE id=? borrando el registro de forma permanente. La interfaz se y la lista de inventario se recarga, ya sin el producto eliminado.

a aquí una captura de pantalla mostrando el cuadro de diálogo que pregunta ¿Deseas eliminar este producto?"}

Confirmar

¿Deseas eliminar este producto?

Sí No

14

Conclusiones

6. Conclusiones

Principales aprendizajes

Durante el desarrollo de este proyecto, el equipo logró consolidar los conocimientos teóricos y prácticos sobre la Programación Orientada a Objetos en Python. La integración funcional entre una interfaz gráfica construida con Tkinter y una base de datos relacional en SQLite nos permitió comprender el ciclo de vida completo de los datos dentro de una aplicación de escritorio. Además, el trabajo colaborativo reforzó nuestras habilidades en la gestión de repositorios con Git y GitHub, facilitando la integración de los aportes de Erick, Ernesto y un servidor en un único entorno de trabajo.

Dificultades enfrentadas

Dado que el proyecto partió de un código base previamente estructurado en clase, uno de los mayores retos fue la refactorización profunda. Adaptar una lógica genérica a las necesidades y al contexto específico del inventario de la cafetería de la UTZin requirió una revisión meticulosa del código para no alterar el funcionamiento del programa al momento de modificar el esquema de la base de datos. Por otro lado, durante las primeras pruebas, notamos que la aplicación era susceptible a cierres inesperados (crasheos) cuando se ingresaban tipos de datos incorrectos o se dejaban campos de texto vacíos.

Soluciones implementadas

Para solucionar los problemas de estabilidad, nos enfocamos en blindar el sistema implementando de manera rigurosa bloques try/except en todas las operaciones CRUD (Agregar, Consultar, Actualizar y Eliminar). Esto garantizó que la aplicación pudiera capturar los errores y mostrar advertencias claras al usuario sin interrumpir la ejecución del programa. Asimismo, para asegurar un código limpio, legible y profesional, realizamos un renombrado sistemático de variables, métodos y clases, de modo que la nomenclatura reflejara fielmente el nuevo modelo de negocio de la cafetería.

Posibles mejoras para futuras versiones

Aunque la aplicación cumple con los requerimientos actuales, el sistema tiene un gran potencial de escalabilidad. Para futuras versiones, se propone implementar:

- **Módulo de seguridad:** Un sistema de inicio de sesión (Login) con diferentes roles (Administrador y Empleado) para restringir el acceso a operaciones críticas como la eliminación de registros.
- **Generación de reportes:** Funcionalidad para exportar el estado del inventario a formatos como PDF o Excel, facilitando los cortes de caja o auditorías de la cafetería.
- **Módulo de estadísticas:** Integración de gráficas visuales que permitan identificar rápidamente los productos con mayor rotación o aquellos que están próximos a agotarse, optimizando así la toma de decisiones del negocio.